Web Technology Lab 07

Multi-Role User System (API + UI + RBAC)

Name: Kunal Sharma
Enrollment No: CS-2341057
Section: 3CSE15 (G1)
GitHub Link:
https://github.com/Roshians/Web-Tech-Lab-06

IILM UNIVERSITY

1 Overview

The objective of this lab is to develop a comprehensive Fullstack Role-Based Access Control (RBAC) system. Using Node.js and Express for the backend and React for the frontend, we implemented a system that distinguishes between three user roles: **SUPER_ADMIN**, **ADMIN**, and **USER**. The system ensures secure authentication using JWT (JSON Web Tokens) and password hashing via bcrypt.

1.1 Roles and Permissions (RBAC Table)

Action	**USER**	**ADMIN**	**SUPER_ADMIN**
Register (Public)	Yes	-	-
Login	Yes	Yes	Yes
View Own Profile (/users/me)	Yes	Yes	Yes
List Users (/users)	No	Yes (USERs only)	Yes (All)
Create User (/users)	No	Yes (USER only)	Yes (USER/ADMIN)
Change Role (/users/:id/role)	No	No	Yes
Delete Users (/users/:id)	No	Yes (USER only)	Yes (Any)

2 API Documentation Table

#	**Method**	**Endpoint**	**Auth**	**Role**	**Fig ID**
1	POST	/api/auth/register	No	Public	Fig-1
2	POST	/api/auth/login	No	Public	Fig-2
3	GET	/api/users/me	Yes	All	Fig-3
4	GET	/api/users	Yes	ADMIN/SUPER	Fig-4
5	POST	/api/users	Yes	ADMIN/SUPER	Fig-5
6	PATCH	/api/users/:id/role	Yes	SUPER_ADMIN	Fig-6
7	DELETE	/api/users/:id	Yes	ADMIN/SUPER	Fig-7

3 API Screenshots

This section contains screenshots of Postman/Thunder Client tests proving the API functionality.

- **Fig-1:** POST /api/auth/register - Registering a new USER account.
- **Fig-2:** POST /api/auth/login - Obtaining JWT token and user profile.
- **Fig-3:** GET /api/users/me - Accessing own profile data using Bearer token.
- **Fig-4:** GET /api/users - ADMIN viewing USER list.
- **Fig-5:** POST /api/users - SUPER_ADMIN creating an ADMIN account.

- **Fig-6:** PATCH /api/users/:id/role - SUPER_ADMIN upgrading a USER to ADMIN.
- **Fig-7:** DELETE /api/users/:id - ADMIN deleting a USER account.

4 RBAC Proof (Forbidden Examples)

Demonstrating server-side enforcement of role restrictions (403 Forbidden).

- **Fig-8:** GET /api/users by USER returns 403 Forbidden (RBAC working).
- **Fig-9:** POST /api/users by ADMIN trying to create an ADMIN returns 403.

5 UI Screenshots

Visual proof of role-based rendering in the React Dashboard.

- **Login Page:** Secure login interface with test account references.
- **USER Dashboard:** Profile view with restricted access to management tools.
- **ADMIN Dashboard:** User management list showing only USER role accounts.
- **SUPER_ADMIN Dashboard:** Full user list with role upgrade and global management tools.

6 Conclusion

Through Lab 07, I gained deep experience in implementing a multi-role system from scratch. Key takeaways include:

- Enforcing RBAC using custom middleware in Express.
- Securing REST APIs with stateless JWT authentication.
- Using React's conditional rendering to display features based on user role.
- Designing professional API documentation and testing for security compliance.